EXPERIMENT 6 — VERSION 1

FEEDFORWARD AND CONVOLUTIONAL NEURAL NETWORKS

AIM

To implement Feedforward Neural Network (FNN) and Convolutional Neural Network (CNN) models using TensorFlow and Keras for image classification using the **Fashion-MNIST and MNIST datasets**.

PROCEDURE

1. Import the required Python libraries.
2. Load the Fashion-MNIST dataset for the FNN.
3. Normalize the image pixel values.
4. Flatten the images into one-dimensional arrays.
5. Build and compile the FNN model.
6. Train and evaluate the FNN model.
7. Load the MNIST dataset for the CNN.
8. Normalize and reshape the images.
9. Build and compile the CNN model using convolution and pooling layers.
10. Train and evaluate the CNN model and display the predictions.

CODE

```
 import numpy as np

import matplotlib.pyplot as plt

import tensorflow as tf


from tensorflow import keras

from tensorflow.keras import layers

from tensorflow.keras.datasets import fashion_mnist, mnist


# FNN using Fashion-MNIST


(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()
x_train_fnn = x_train.reshape(-1, 784).astype("float32") /

255.0 x_test_fnn = x_test.reshape(-1, 784).astype("float32") /
```

```
255.0

fnn = keras.Sequential([
 layers.Input(shape=(784,)),
 layers.Dense(128, activation="relu"),
 layers.Dense(64, activation="relu"),
 layers.Dense(10, activation="softmax")
])

fnn.compile(
 optimizer="adam",
 loss="sparse_categorical_crossentropy",
 metrics=["accuracy"]
)

fnn.fit(
 x_train_fnn,
 y_train,
 epochs=5,
 batch_size=128,
 validation_split=0.1
)

loss_fnn, accuracy_fnn = fnn.evaluate(
 x_test_fnn,
 y_test,
 verbose=0
)

print("FNN Test Loss:", loss_fnn)
```

```
print("FNN Test Accuracy:", accuracy_fnn)



# CNN using MNIST

(x_train, y_train), (x_test, y_test) = mnist.load_data()

x_train_cnn = x_train.reshape(-1, 28, 28, 1).astype("float32") /
255.0 x_test_cnn = x_test.reshape(-1, 28, 28, 1).astype("float32") /
255.0

cnn = keras.Sequential([
 layers.Input(shape=(28, 28, 1)),
 layers.Conv2D(32, (3, 3), activation="relu"),
 layers.MaxPooling2D((2, 2)),
 layers.Conv2D(64, (3, 3), activation="relu"),
 layers.MaxPooling2D((2, 2)),
 layers.Flatten(),
 layers.Dense(128, activation="relu"),
 layers.Dense(10, activation="softmax")
])

cnn.compile(
 optimizer="adam",
 loss="sparse_categorical_crossentropy",
 metrics=["accuracy"]
)

cnn.fit(
 x_train_cnn,
```

```
 y_train,
 epochs=5,
 batch_size=128,
 validation_split=0.1
)

loss_cnn, accuracy_cnn = cnn.evaluate(
 x_test_cnn,
 y_test,
 verbose=0
)

print("CNN Test Loss:", loss_cnn)
print("CNN Test Accuracy:", accuracy_cnn)



# Sample Prediction


prediction = cnn.predict(x_test_cnn[:10], verbose=0)
predicted = np.argmax(prediction, axis=1)


print("Actual Labels :", y_test[:10])
print("Predicted Labels:", predicted)
# Display images


plt.figure(figsize=(10, 5))


for i in range(10):
 plt.subplot(2, 5, i + 1)
 plt.imshow(x_test[i], cmap="gray")
```

```
plt.title("Actual: " + str(y_test[i]) +
"\nPred: " + str(predicted[i]))
plt.axis("off")

plt.tight_layout()
plt.show()
```

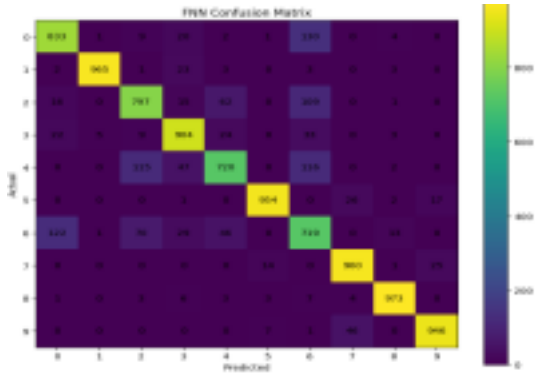
OUTPUT:

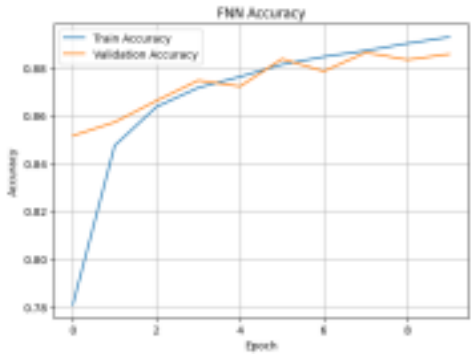
```
========== EVALUATING FNN ==========
FNN Test Loss: 0.3352
FNN Test Accuracy: 0.8771

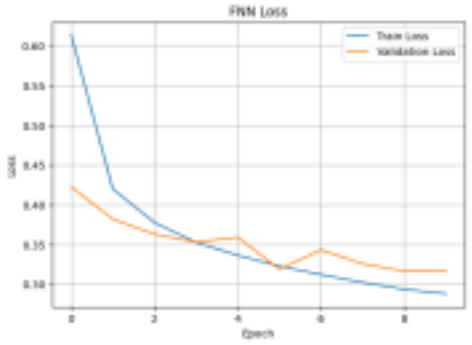
========== FNN CLASSIFICATION REPORT ==========
              precision    recall  f1-score   support

           0       0.84      0.83      0.83      1000
           1       0.99      0.96      0.98      1000
           2       0.79      0.80      0.80      1000
           3       0.87      0.90      0.88      1000
           4       0.84      0.72      0.77      1000
           5       0.97      0.95      0.96      1000
           6       0.64      0.72      0.68      1000
           7       0.93      0.96      0.94      1000
           8       0.97      0.97      0.97      1000
           9       0.96      0.95      0.95      1000

    accuracy                           0.88     10000
   macro avg       0.88      0.88      0.88     10000
weighted avg       0.88      0.88      0.88     10000
```





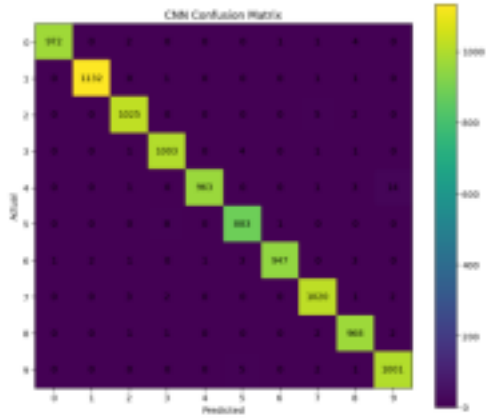


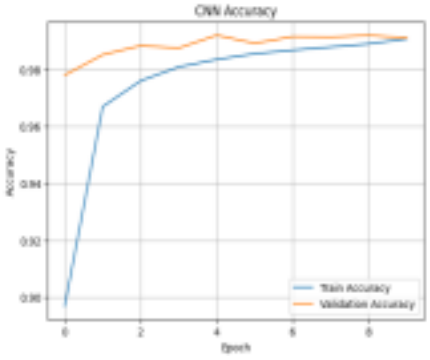

```
---------- EVALUATING CNN ----------
CNN Test Loss: 0.0238
CNN Test Accuracy: 0.9914

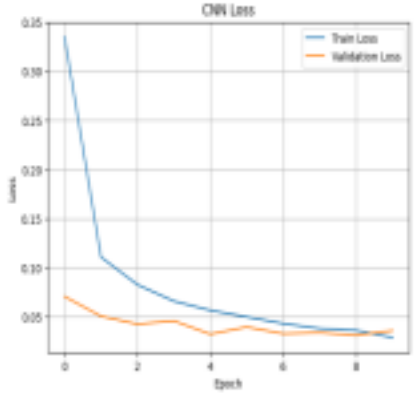
---------- CNN CLASSIFICATION REPORT ----------
              precision    recall  f1-score   support

           0       1.00      0.99      1.00       980
           1       1.00      1.00      1.00      1135
           2       0.99      0.99      0.99      1032
           3       0.99      0.99      0.99      1010
           4       1.00      0.98      0.99       982
           5       0.99      0.99      0.99       892
           6       1.00      0.99      0.99       958
           7       0.99      0.99      0.99      1028
           8       0.98      0.99      0.99       974
           9       0.98      0.99      0.99      1009

    accuracy                           0.99     10000
   macro avg       0.99      0.99      0.99     10000
weighted avg       0.99      0.99      0.99     10000
```









RESULT

Thus, the **Feedforward Neural Network and Convolutional Neural Network** were successfully implemented using Fashion-MNIST and MNIST datasets. The CNN achieved better performance for image classification because convolution and pooling layers effectively extract spatial features from images.